

TestBento Application Documentation

1. Project Overview

TestBento is a comprehensive Quality Assurance (QA) Test Management application designed to streamline the software testing lifecycle. It provides a centralized platform for QA teams, developers, and managers to collaborate on testing efforts, from requirement analysis to test execution and defect tracking.

2. Purpose of the Application

The primary purpose of TestBento is to ensure software quality by offering a structured workflow for:

- Organizing requirements into Epics, Features, and User Stories.
- Writing, reviewing, and managing Test Cases.
- Executing test cycles and tracking results.
- Logging and resolving defects.
- Providing real-time analytics and Requirement Traceability Matrix (RTM) to ensure comprehensive test coverage.

3. Tech Stack

- **Frontend Framework**: Next.js 14 (App Router)
- **UI Library**: React 18
- **Styling**: Tailwind CSS
- **State Management**: Zustand
- **Database & Authentication**: Supabase (PostgreSQL)
- **Charts & Analytics**: Recharts
- **Icons**: Lucide React
- **AI Integration**: Google Generative AI (Gemini) for automated test case generation

4. Role-Based Access Control (RBAC)

TestBento implements a strict RBAC system to ensure data security and proper workflow enforcement:

- **ADMIN**: Full access to all modules, project creation, and user management.
- **QA LEAD**: Can create projects, manage hierarchy, review/approve test cases, create execution cycles, and assign tasks.
- **QA ENGINEER**: Can write test cases, execute assigned tests, log defects, and submit test cases for review.
- **DEVELOPER**: Can view defects and update their status.

- **MANAGER**: Read-only access to high-level reports and analytics.
- **VIEWER**: Basic read-only access to the project.

5. Project Modules & Features

5.1. Dashboard

- **Features**: Real-time analytics, KPIs, project execution trends, and automation coverage.
- **Usage**: Provides a bird's-eye view of the workspace or specific projects. Managers use this to monitor overall health and pass/fail rates.

5.2. Projects

- **Features**: Project creation, renaming, and deletion.
- **Usage**: The starting point. Groups all related epics, test cases, and executions under a specific product or release.

5.3. Hierarchy (Requirements)

- **Features**: Manage Epics, Features, and User Stories.
- **Usage**: Structures the requirements. Every test case must be linked to a User Story to ensure traceability.

5.4. Test Cases

- **Features**: Create, edit, and organize test steps, expected results, priority, and automation status. Includes an AI-powered generator to create test cases from requirements.
- **Usage**: QA Engineers use this module to define the exact steps to validate a User Story.

5.5. Reviews

- **Features**: Peer review workflow for test cases.
- **Usage**: Ensures test quality. A QA Engineer submits a test case; a QA Lead reviews and marks it as `APPROVED` or `REJECTED`. Only approved test cases can be executed.

5.6. Execution

- **Features**: Test cycles (e.g., Sprints, Releases), execution assignments, and status tracking (Pass, Fail, Blocked, Not Run).
- **Usage**: QA Leads create cycles and assign test cases. QA Engineers execute them and update the status based on actual software behavior.

5.7. Defects

- **Features**: Bug logging, severity assignment (Critical, High, Medium, Low), and status tracking (Open, In Progress, Resolved, Closed).
- **Usage**: Used to track issues found during execution. Can be linked directly to a test case and execution item.

5.8. RTM (Requirement Traceability Matrix)

- **Features**: Visual mapping of User Stories to Test Cases and their execution status.
- **Usage**: Ensures every requirement has corresponding test cases and shows whether those tests have passed, highlighting untested areas.

5.9. Reports

- **Features**: Visual charts (Bar, Pie, Area) for execution status, cycle comparison, test case distribution, and defect density.
- **Usage**: Deep dive analytics for managers and leads to assess release readiness.

5.10. Members

- **Features**: Manage project-specific access.
- **Usage**: Add or remove users from a project and assign them roles.

5.11. Audit Logs

- **Features**: Detailed tracking of user actions (Create, Update, Delete) across the project.
- **Usage**: Security and accountability. Tracks who changed what and when.

6. Project Lifecycle (Proper Flow)

To use TestBento effectively, a new user should follow this standard workflow:

1. **Setup Phase**
 - **Admin/QA Lead** creates a new **Project**.
 - **Admin/QA Lead** goes to the **Members** module and adds team members to the project.
2. **Requirements Phase**
 - **QA Lead** navigates to the **Hierarchy** module.
 - Creates an **Epic** -> adds **Features** -> adds **User Stories** with clear Acceptance Criteria.
3. **Test Design Phase**
 - **QA Engineer** goes to the **Test Cases** module.
 - Writes test cases for the created User Stories (or uses the AI Generate feature).
 - **QA Engineer** navigates to the **Reviews** module and submits their Draft test cases to a QA Lead for review.
4. **Review Phase**

- *QA Lead* receives a notification, reviews the test cases in the **Reviews** module, and approves them.

5. **Execution Planning Phase**

- *QA Lead* goes to the **Execution** module and creates a new Execution Cycle (e.g., "Sprint 1 Regression").

- Adds the *Approved* test cases to this cycle and assigns them to QA Engineers.

6. **Execution & Defect Tracking Phase**

- *QA Engineer* runs the tests in the application being tested.

- Updates the status in the **Execution** module to `PASS`, `FAIL`, or `BLOCKED`.

- If a test fails, the *QA Engineer* navigates to the **Defects** module to log a bug, linking it to the test case.

7. **Resolution Phase**

- *Developer* fixes the bug and updates the defect status to `RESOLVED`.

- *QA Engineer* re-runs the failed test case and updates it to `PASS`.

8. **Reporting Phase**

- *Manager/QA Lead* checks the **Dashboard**, **Reports**, and **RTM** modules to ensure 100% coverage and a high pass rate before release.